of a Bloom filter with a storage capacity of 8 bits and then try to store 1 million objects in it, chances are that you won't get great performance. On the contrary, considering a Bloom filter with an 8-but buffer, its whole buffer would be set to `1` after approximately 10-20 hashes: at that point, all calls to `contains` will just return `true`, and you will not be able to understand if an object was actually stored in the container or not.

If, instead, we allocate sufficient space and choose our hash functions well, then the indices generated for each key won’t clash, and for any two different keys the overlap between the lists of indices generated will be minimal, if any.

But how much space is sufficient space? Internally a Bloom filter translates each key into a sequence of k indices chosen out of m possible alternatives[16]: the trick is that we store keys efficiently by flipping these k bits in the filter’s buffer, a bit-array.

If you have brushed up your school algebra recently, you might have guessed that we can only represent m^k different sequences of k elements drawn from m values; we need, however, to make two considerations:

1. We actually can't even use all these sequences: we would like all indices associated to a key to be different (otherwise we would actually store less than k bits per key) and so we strive for all lists of k indices to be duplicate-free;
2. We are not really interested in the order of these k indices, it’s completely irrelevant if we first write the bit